StateShift

Move a running workload between machines.

No restart. No repeated work.

github.com/guneettoppo/stateshift

TEAM · NAME · NAME · NAME

01 / THE FAILURE

The failure you have already lived through

AGENT RUN

40 tool calls, $6 of tokens, the box dies at call 31.



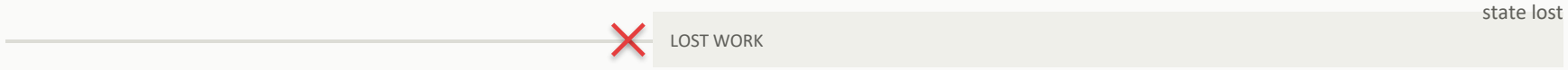
SIMULATION

Six hours of progress held in memory. Restarting means six hours again.



LIVE SESSION

Unpersisted state, open handles, work in flight — no “just redeploy it”.



Restarting is not just slow — it pays twice, for tokens, for compute, for the same work.

02 / WHY THE OBVIOUS ANSWERS FAIL

Each one solves part of it, none moves the job

ANSWER		WHY IT MISSES
Redeploy on the new node	✕	Works for stateless services only. Loses in-memory progress and duplicates committed side effects.
Persist after every step	✕	That is durable execution — it keeps the job pinned to one machine, and it dies with the machine.
CRIU / process migration	✕	Needs root, matching kernels, matching images. All-or-nothing inside 24 hours.
VM live migration	✕	Solves it at the hypervisor layer, which means you own the hypervisor.

Chosen instead: move the workload’s explicit state at a step boundary, with a database-enforced handover.

03 / THE IDEA

Checkpoint what you own, move what changed, fence who may write

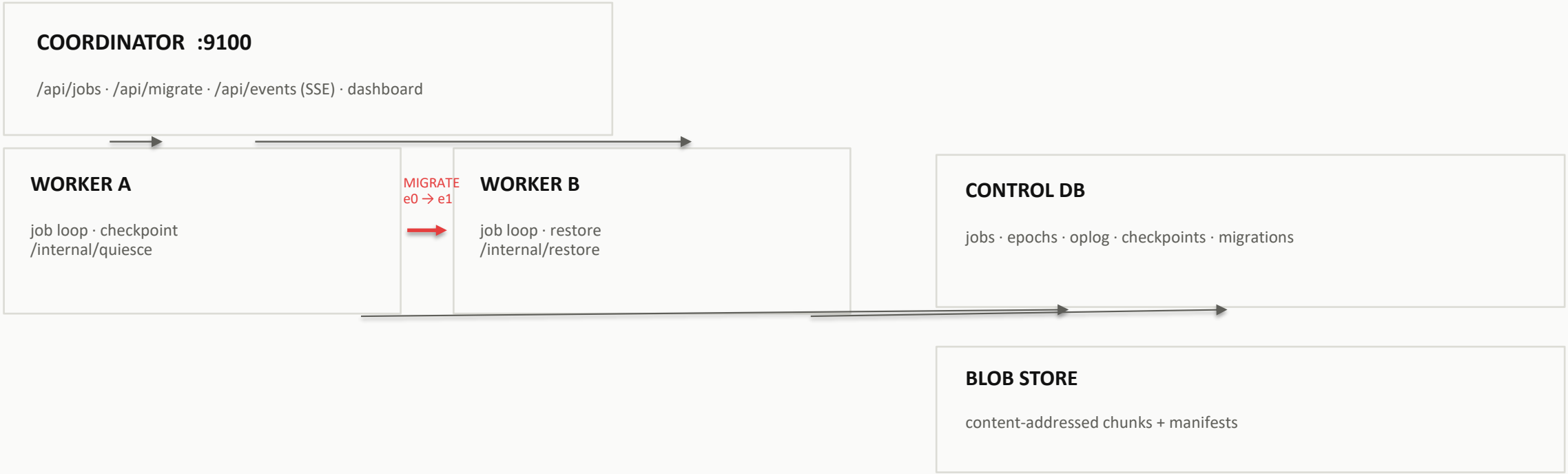
The workload is a state machine. We move its state at a step boundary and hand ownership with one atomic statement.



The trick is not moving memory — it is deciding, atomically, who is allowed to write.

04 / ARCHITECTURE

Three roles. The coordinator drives, the database decides.



Deployment rule: the control plane and the database never live on a worker — kill a node without killing the demo.

05 / CONSISTENCY

The fence: one guarded statement decides who may write

```
UPDATE jobs
   SET epoch  = epoch + 1,
       owner  = $target,
       status = 'restoring'
 WHERE id     = $job
   AND epoch = $current;

-- rows affected must be exactly 1, otherwise abort
```

WHY A HEARTBEAT IS NOT ENOUGH

A heartbeat answers “is it alive?” with a guess, hundreds of milliseconds late.
This answers “who may write?” with a predicate evaluated at commit.

Two nodes can both think they own the job. Only one can commit — ownership is a row, not a heartbeat.

06 / EXACTLY-ONCE

Effects and progress commit in one transaction

01 Every step produces effects with stable IDs.

02 The effect log and the state advance commit together.

03 A replay is a no-op by schema, not by policy.

04 Result: 800 ops for 800 steps after a mid-flight migration.

```
BEGIN;
INSERT INTO oplog (job_id, seq, epoch, op_id, result)
  VALUES ($job,$seq,$epoch,$op_id,$result)
  ON CONFLICT (job_id,seq,op_id) DO NOTHING;
UPDATE jobs SET seq = $next, state = $state
 WHERE id = $job AND epoch = $epoch AND seq = $from;
COMMIT;   -- 0 rows updated => stale, abort
```

Duplication becomes impossible, not unlikely.

07 / TRANSFER EFFICIENCY

The second checkpoint ships one chunk out of seven

1 / 7

chunks re-uploaded on the second checkpoint of the same job



6 skipped · 1 uploaded

12.4 KiB state → 1,268 B of 12,377 B moved.

Content-defined chunking, ~1 KiB average, sha256-addressed.
Chunks the store already holds are skipped, never re-sent.
About 10% of the bytes — the requirement, as one number.

Immutable head, fixed-width tail: if a counter changes length, every boundary shifts and dedupe collapses to 0%.

08 / LIVE DEMO

Same job, same sequence number, different machine

Five numbers on the dashboard, in the order that matters.

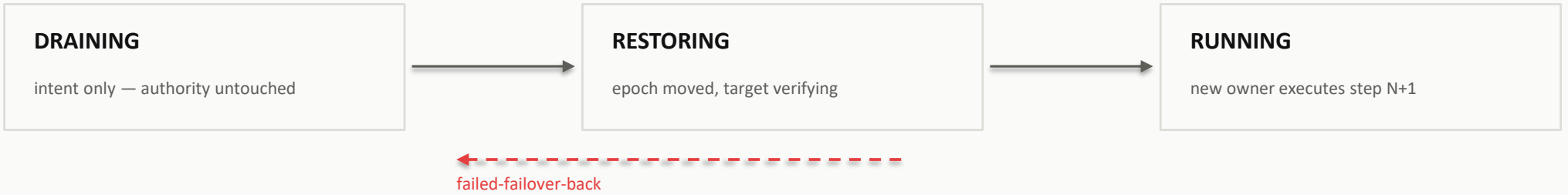
EPOCH	SEQ	OPS	BYTES	PAUSE
0 → 1	800 / 800	800	13,645	3 ms

QUIESCING → FENCING → RESTORING

Owner changes, epoch changes, the job does not restart. Then we kill the source and show it again.

09 / FAILURE HANDLING

Every failure path ends with exactly one owner



Dies before the fence	source keeps running, migration aborts, nothing lost	aborted-quiesce-failed
Dies after the fence, before restore	authority returns to the source with a new epoch	failed-failover-back
Both alive, network partitioned	only the epoch holder can commit; the other is rejected	ok

Every attempt is recorded — including the failures. The red rows are the ones that prove the design.

10 / PROOF

The correctness proof, not a screenshot

ops == steps **800 / 800**

checksum == control **e2a580b9471db5b1**

pause = **3 ms (loopback, 12 KiB state)**

A migrated run and a run that never migrated end at the same checksum.

One committed op per step, zero duplicates, identical final state.

11 / LIMITATIONS

What this is not

> We move application state — not sockets, file descriptors or kernel memory.

> Workloads must express progress as a step machine: Init, Step, Checksum.

> A node dying with no migration in flight is not yet auto-recovered (~2 h of work).

Transparent process migration is a different problem with different constraints. We chose the version that can be correct in a day.

12 / NEXT

The next hour, the next week

01	**Watchdog**	recover a dead owner automatically — designed, not built
02	**Streaming transfer**	ship chunks during the step instead of upload-then-download
03	**Out-of-process SDK**	run third-party workloads under the same fence

The next hour of work is the watchdog. The next week is making it a platform.

One machine to another, mid-run, without paying twice.

StateShift moves a running workload between machines and proves, with a fence and an op log, that nothing was repeated.

YugabyteDB **DigitalOcean** **Gemini** **Tiger Data**

github.com/guneettoppo/stateshift